



Digital Design

Chapter 3: Sequential Logic Design -- Controllers

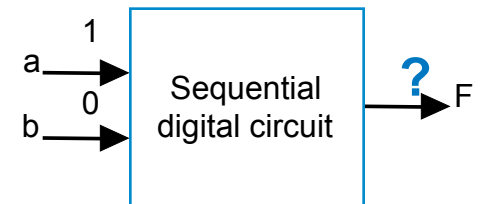
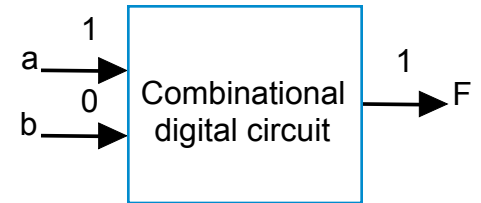
Slides to accompany the textbook *Digital Design*, First Edition,
by Frank Vahid, John Wiley and Sons Publishers, 2007.
<http://www.ddvahid.com>

Copyright © 2007 Frank Vahid

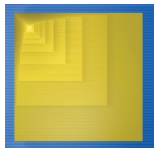
Instructors of courses requiring Vahid's *Digital Design* textbook (published by John Wiley and Sons) have permission to modify and use these slides for customary course-related activities, subject to keeping this copyright notice in place and unmodified. These slides may be posted as unanimated pdf versions on publicly-accessible course websites.. PowerPoint source (or pdf with animations) may not be posted to publicly-accessible websites, but may be posted for students on internal protected sites or distributed directly to students by other electronic means. Instructors may make printouts of the slides available to students for a reasonable photocopying charge, without incurring royalties. Any other use requires explicit permission. Instructors may obtain PowerPoint source or obtain special use permissions from Wiley – see <http://www.ddvahid.com> for information.

Introduction

- Sequential circuit
 - Output depends not just on present inputs (as in combinational circuit), but on past sequence of inputs
 - Stores bits, also known as having “state”
 - Simple example: a circuit that counts up in binary
- In this chapter, we will:
 - Design a new building block, a **flip-flop**, that stores one bit
 - Combine that block to build multi-bit storage – a **register**
 - Describe the sequential behavior using a **finite state machine**
 - Convert a finite state machine to a **controller** – a sequential circuit having a register and combinational logic

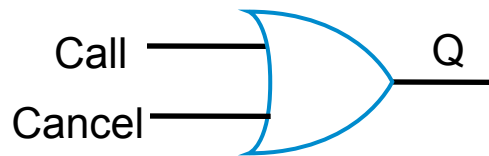


*Must know
sequence of past
inputs to know
output*



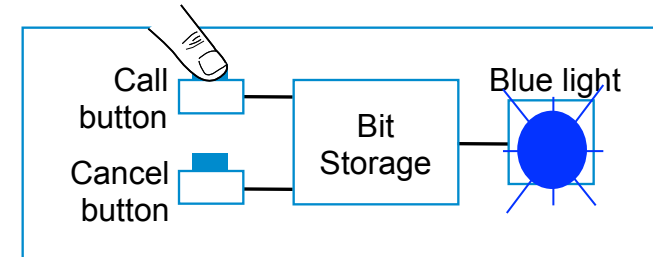
Example Needing Bit Storage

- Flight attendant call button
 - Press call: light turns on
 - **Stays on** after button released
 - Press cancel: light turns off
 - Logic gate circuit to implement this?

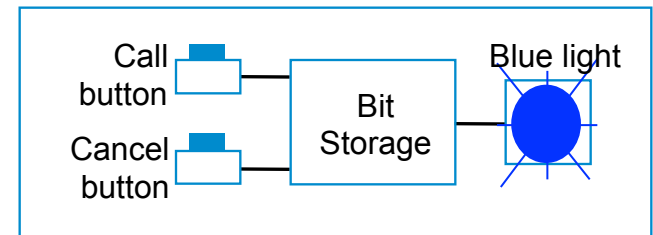


Doesn't work. $Q=1$ when $Call=1$, but doesn't stay 1 when $Call$ returns to 0

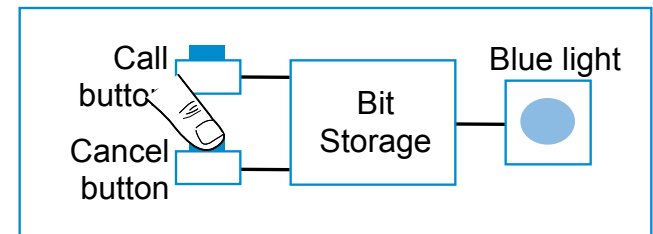
Need some form of "feedback" in the circuit



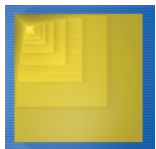
1. Call button pressed – light turns on



2. Call button released – light **stays on**

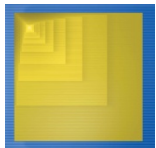
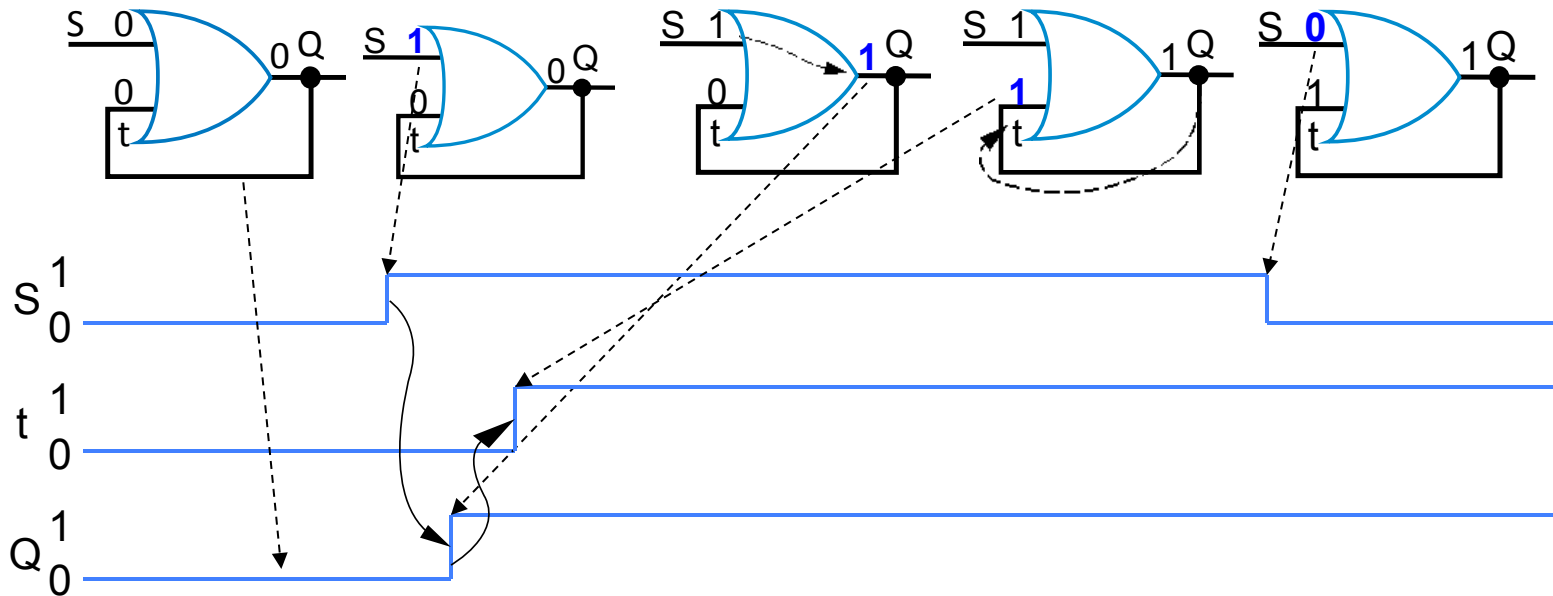
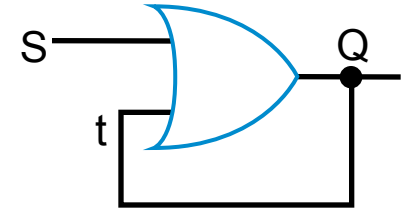


3. Cancel button pressed – light turns off



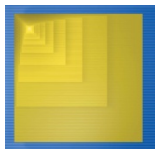
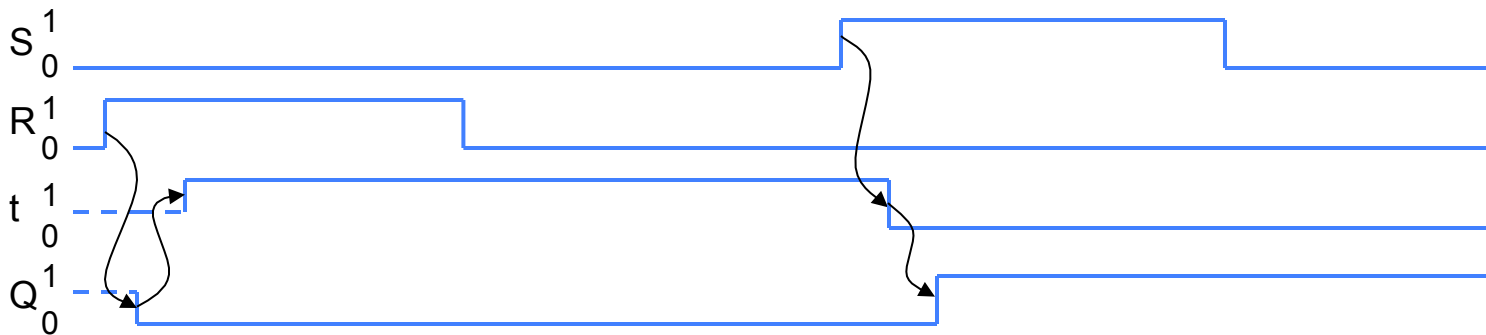
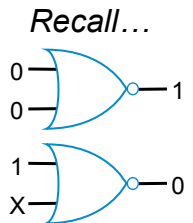
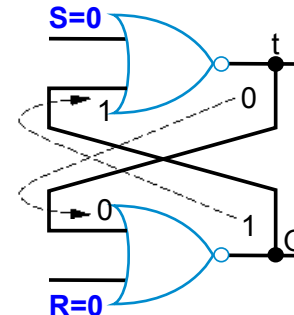
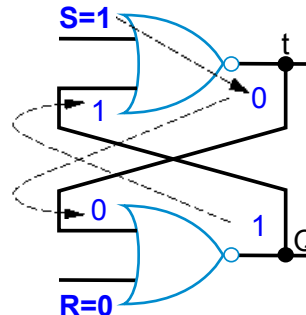
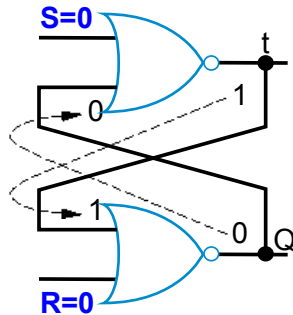
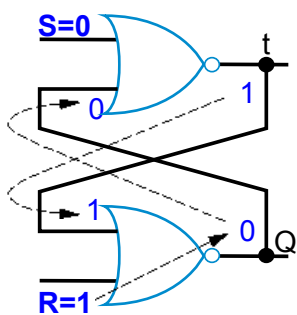
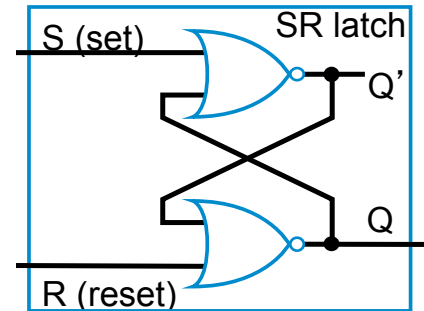
First attempt at Bit Storage

- We need some sort of feedback
 - Does circuit on the right do what we want?
 - No: Once Q becomes 1 (when S=1), Q stays 1 forever – no value of S can bring Q back to 0



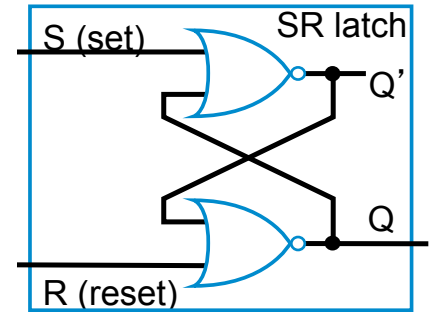
Bit Storage Using an SR Latch

- Does the circuit to the right, with cross-coupled NOR gates, do what we want?
 - Yes! How did someone come up with that circuit? Maybe just trial and error, a bit of insight...

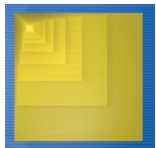


SR Latch

- Q' is usually the complement of Q .
- S sets or presets the Q output to 1.
- R resets or clears the Q output to 0.

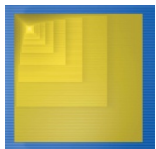
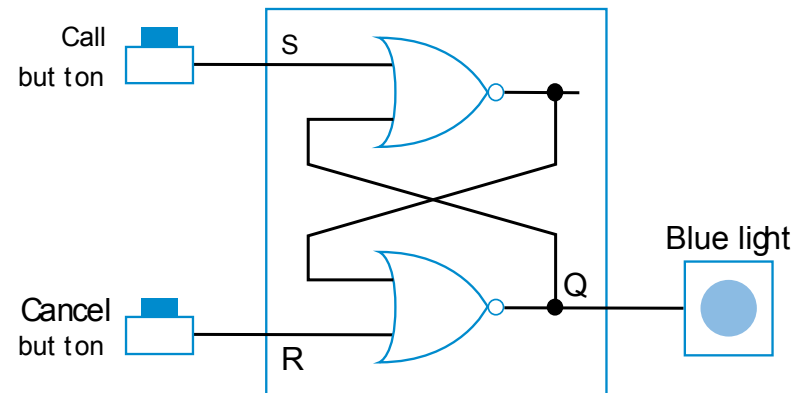
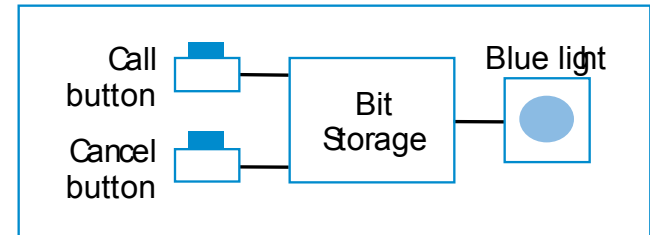


S	R	Q	Q'
0	0	last Q	last Q'
0	1	0	1
1	0	1	0
forbidden	1	1	0



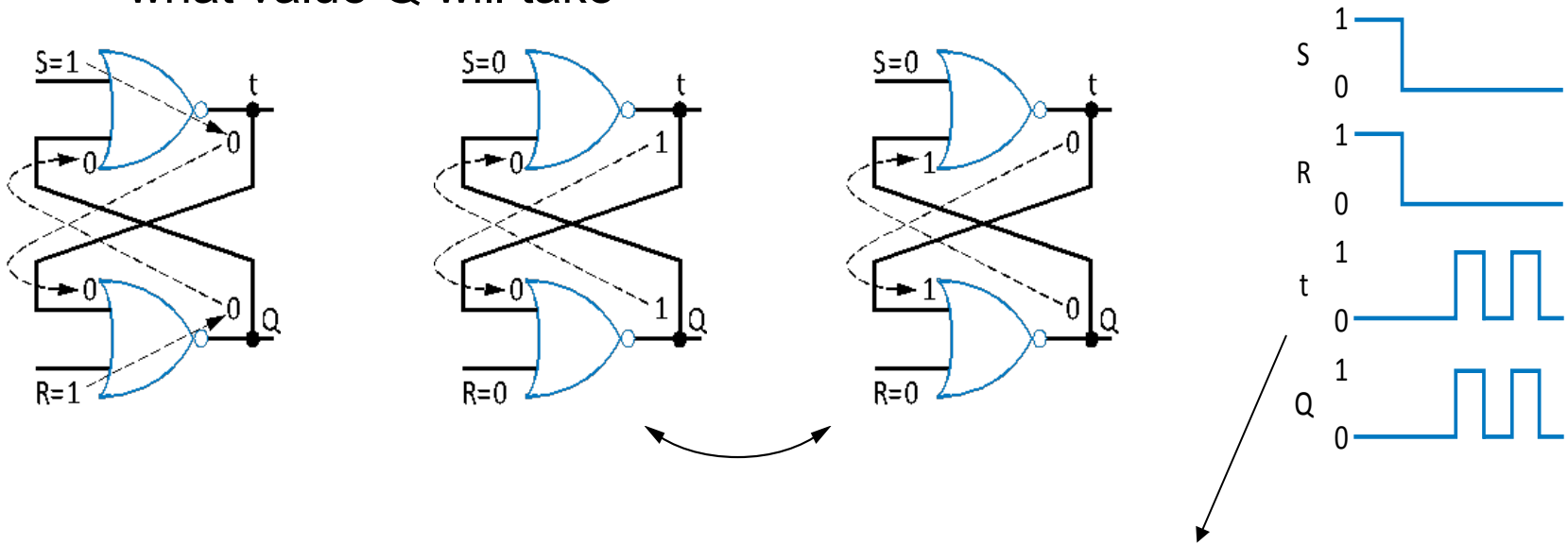
Example Using SR Latch for Bit Storage

- SR latch can serve as bit storage in previous example of flight-attendant call button
 - Call=1 : sets Q to 1
 - Q stays 1 even after Call=0
 - Cancel=1 : resets Q to 0
- But, there's a problem...



Problem with SR Latch

- Problem
 - If $S=1$ and $R=1$ simultaneously, followed by $S=R=0$, we don't know what value Q will take

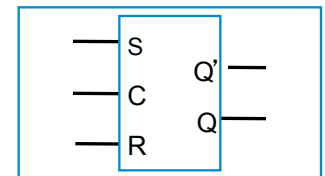
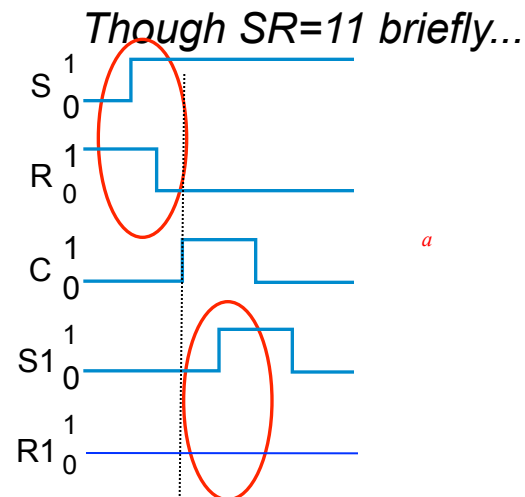
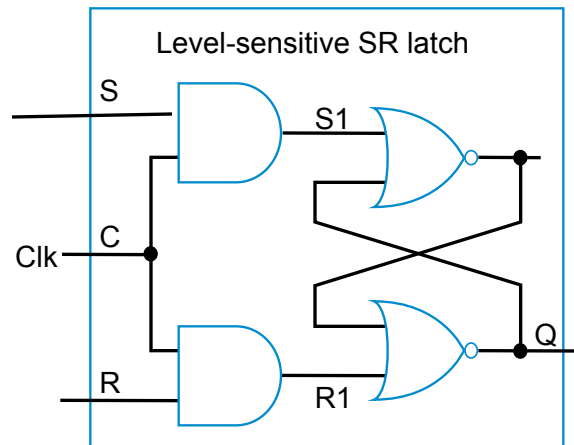
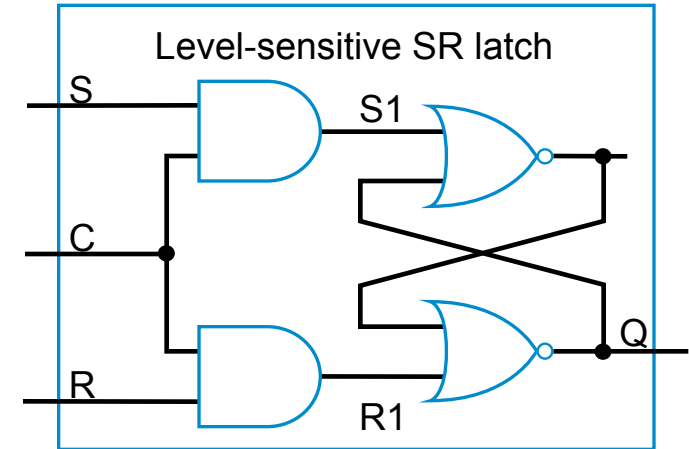


Q may oscillate. Then, because one path will be slightly longer than the other, Q will eventually settle to 1 or 0 – but we don't know which.

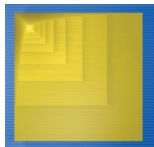
Known as a race condition

Solution: Level-Sensitive SR Latch

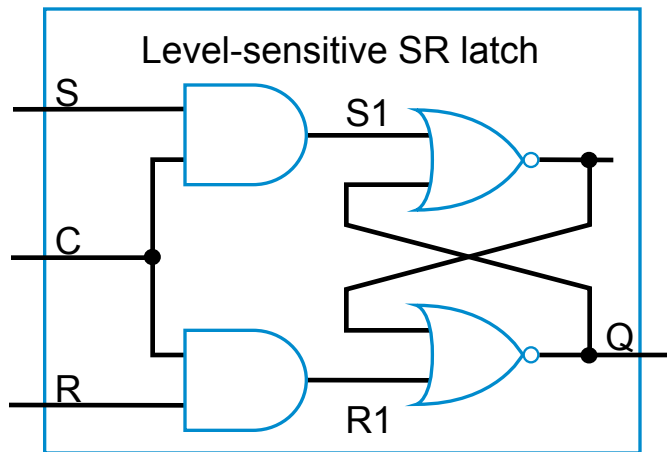
- Add enable input “C” as shown
 - Only let S and R change when C=0
 - Ensure circuit in front of SR never sets $SR=11$, except briefly due to path delays
 - Change C to 1 only after sufficient time for S and R to be stable
 - When C becomes 1, the stable S and R value passes through the two AND gates to the SR latch’s S1 R1 inputs.



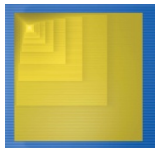
Level-sensitive SR latch symbol



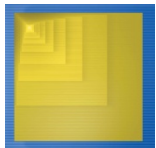
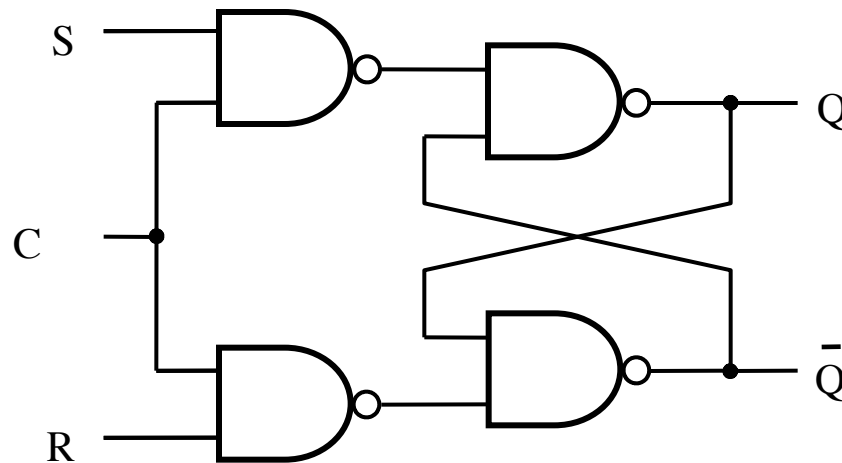
Level-Sensitive SR Latch



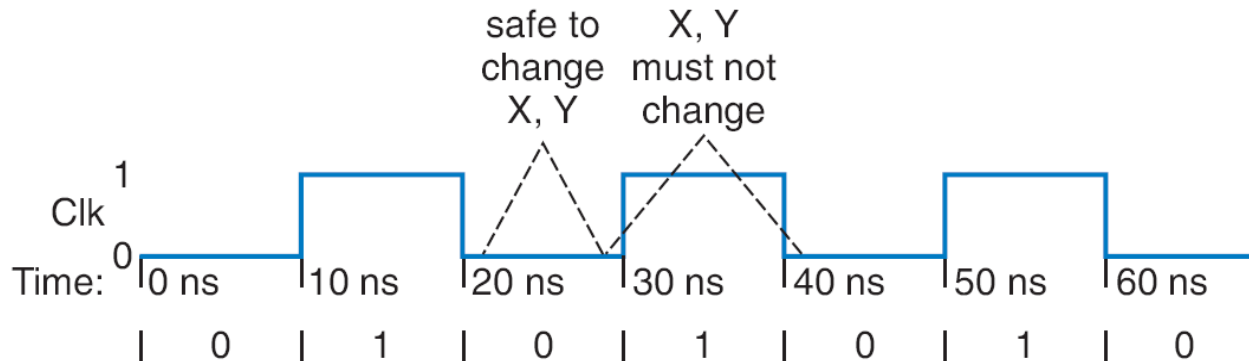
C	S	R	$Q(t+1)$
0	x	x	$Q(t)$ (no change)
1	0	0	$Q(t)$ (no change)
1	0	1	0
1	1	0	1
1	1	1	x



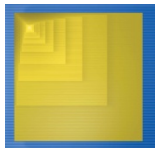
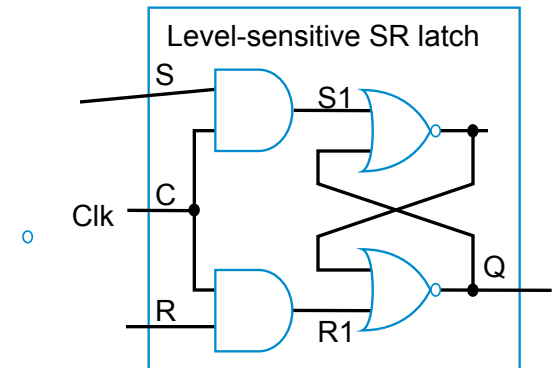
Level-Sensitive SR Latch with NAND gates



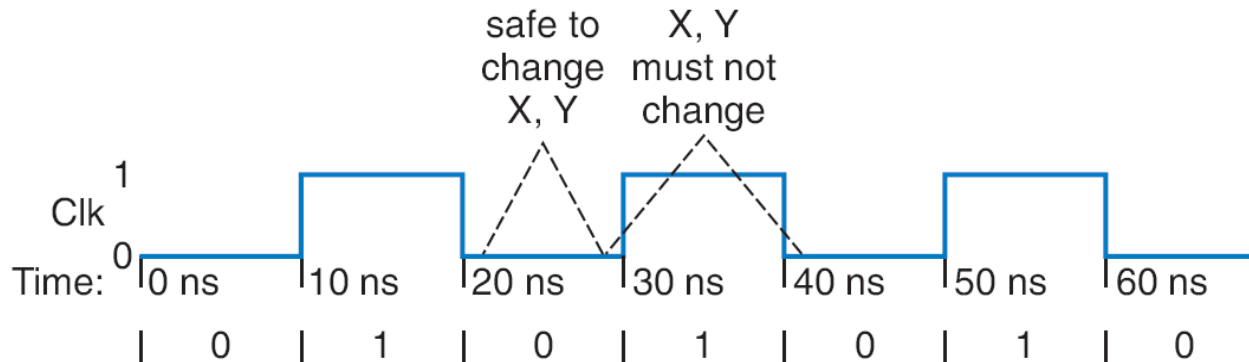
Clock Signals for a Latch



- How do we know when it's safe to set $C=1$?
 - Most common solution – make C pulse up/down
 - $C=0$: Safe to change X, Y
 - $C=1$: Must *not* change X, Y
 - We'll see how to ensure that later
 - **Clock** signal -- Pulsing signal used to enable latches
 - Because it ticks like a clock
 - Sequential circuit whose storage components all use clock signals: **synchronous** circuit
 - Most common type
 - Asynchronous circuits – important topic, but left for advanced course

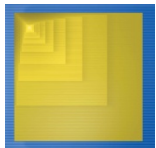


Clocks



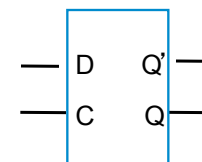
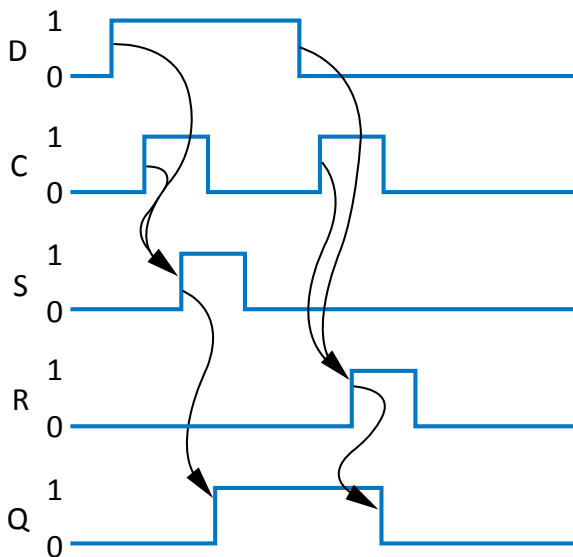
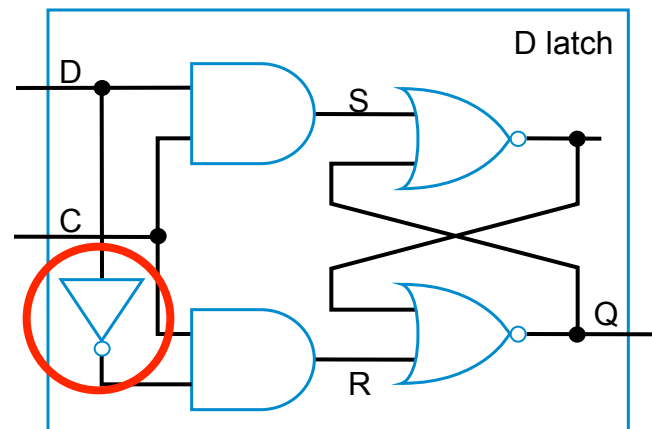
- **Clock period:** time interval between pulses
 - Above signal: period = 20 ns per cycle
- **Clock cycle:** duration of 1 period
 - Above signal shows 3.5 clock cycles
- **Clock frequency:** $1/\text{period}$
 - Above signal period: 20 ns per cycle
 - frequency = $1 / 20 \text{ ns} = 50 \text{ MHz}$
 - 1 Hz = 1/second (1 cycle per second)

Freq	Period
100 GHz	0.01 ns
10 GHz	0.1 ns
1 GHz	1 ns
100 MHz	10 ns
10 MHz	100 ns



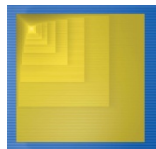
Level-Sensitive D Latch

- Level sensitive SR latch requires careful design to ensure $SR=11$ never occurs
- D latch relieves designer of that burden
 - Inserted inverter ensures R always opposite of S

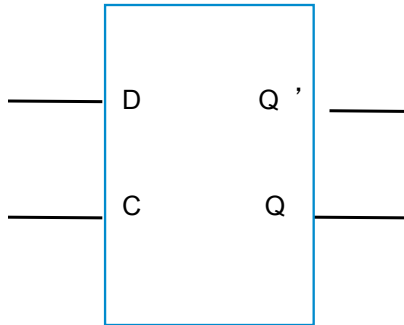


D latch symbol

(also known as
transparent latch)

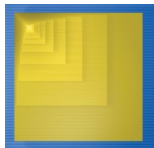


Level-Sensitive D Latch



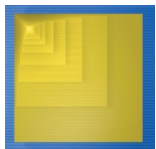
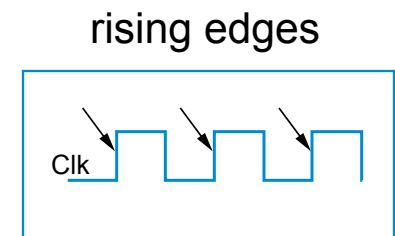
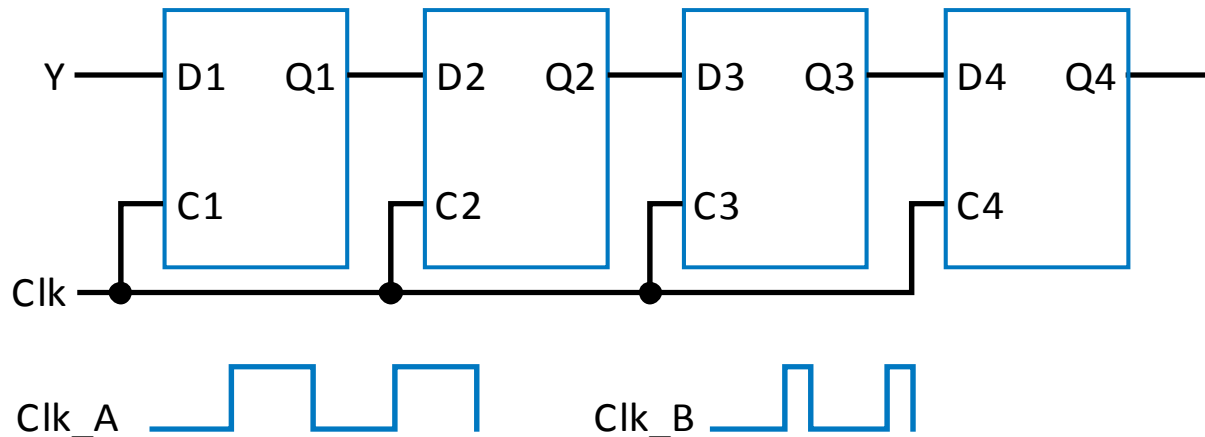
Clk	D	$Q(t + 1)$
0	x	$Q(t)$
1	0	0
1	1	1

There is a problem with Level-Sensitive D Latch when setup, hold times are violated, leading to oscillation. D needs to be stable during setup and hold times. Will discuss at the end of this chapter.

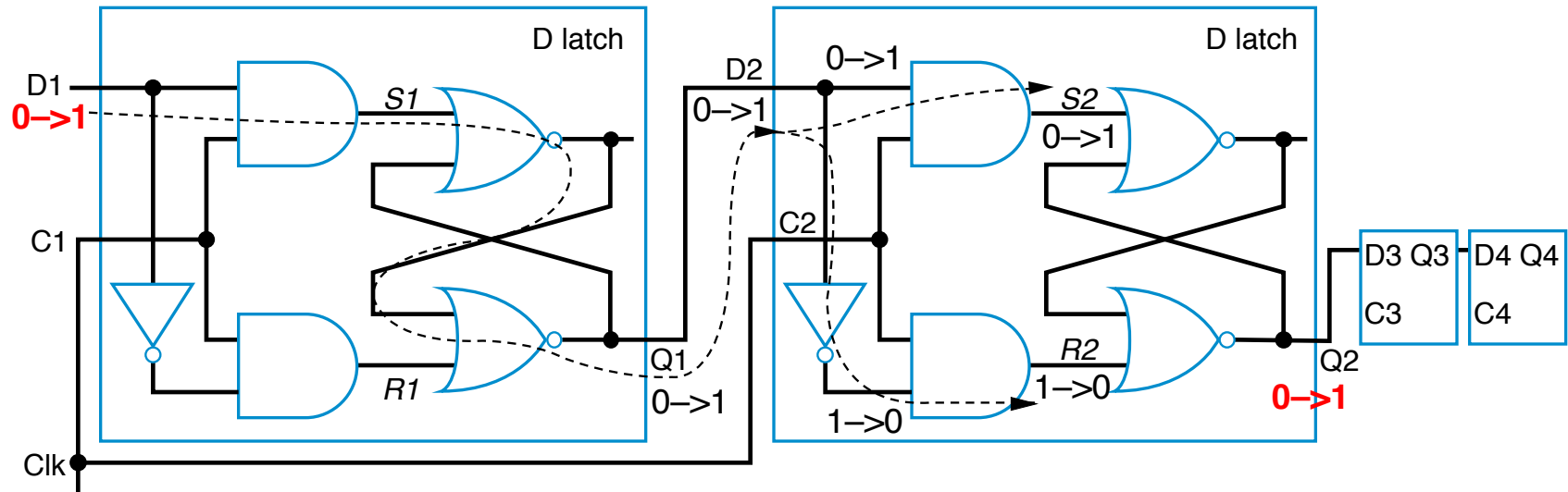


Problem with Level-Sensitive D Latch

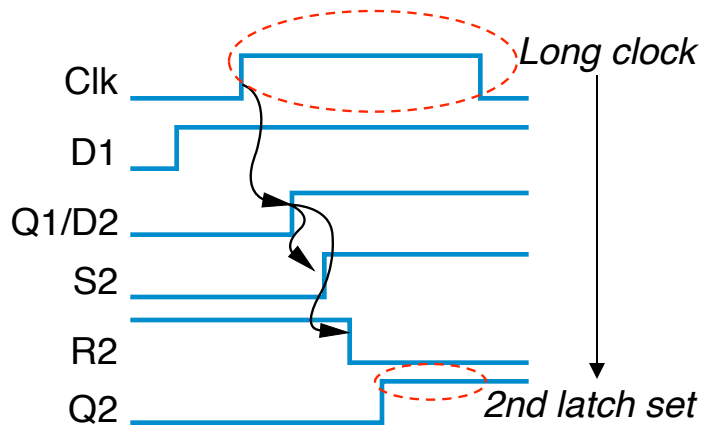
- D latch still has problem (as does SR latch)
 - When $C=1$, through how many latches will a signal travel?
 - Depends on for how long $C=1$
 - Clk_A -- signal may travel through multiple latches
 - Clk_B -- signal may travel through fewer latches
 - Hard to pick C that is just the right length
 - Can we design bit storage that only stores a value on the rising edge of a clock signal? That is, at the instant clock changes.



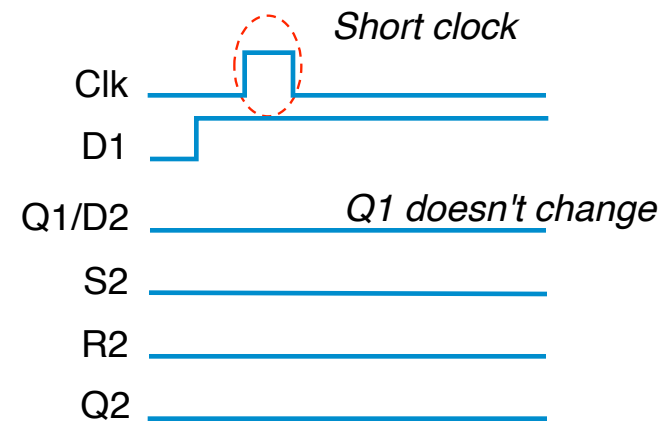
Problem with Level-Sensitive D Latch



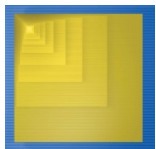
(a)



(b)

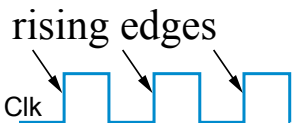


(c)

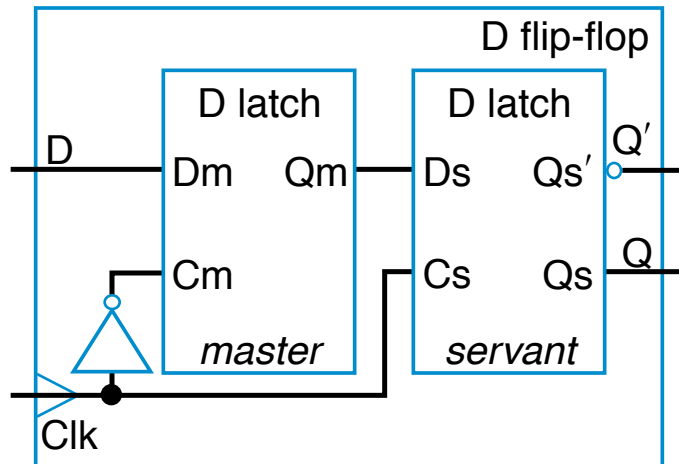


D Flip-Flop

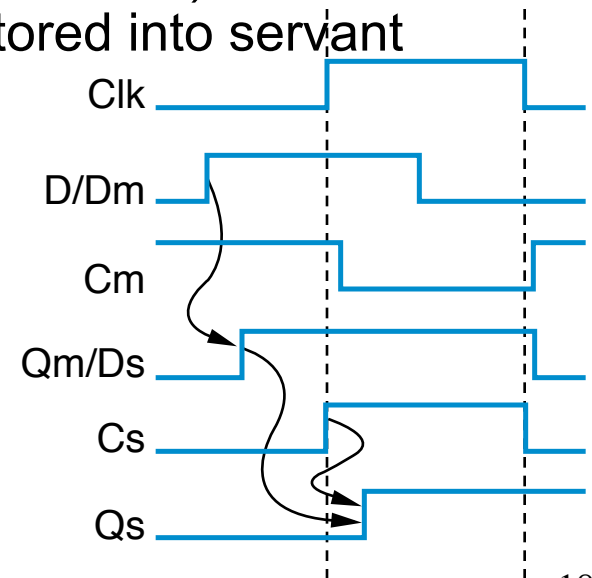
Can we design bit storage that only stores a value on the rising edge of a clock signal?



- **Flip-flop:** Bit storage that stores on clock edge
- One design – master-servant
 - Clk = 0 – master enabled, loads D, appears at Qm. Servant disabled.
 - Clk = 1 – Master disabled, Qm stays same. Servant latch enabled, loads Qm, appears at Qs.
 - Thus, value at D (and hence at Qm) when Clk changes from 0 to 1 gets stored into servant

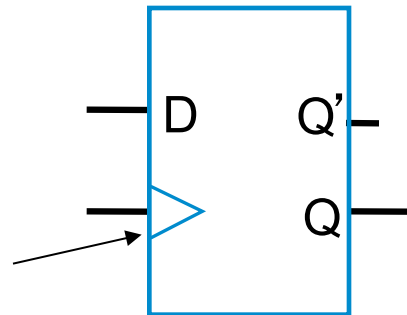


Note:
Hundreds
of different
flip-flop
designs
exist

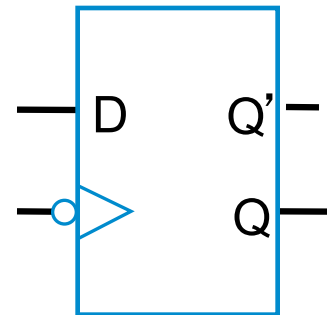
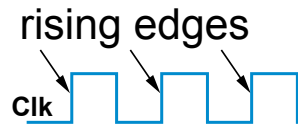


D Flip-Flop

The triangle means clock input, edge triggered

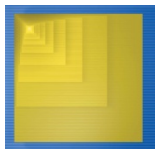
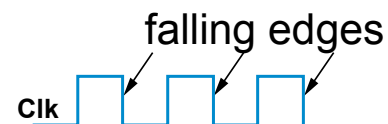


Symbol for rising-edge triggered D flip-flop



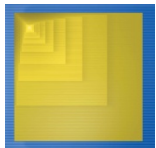
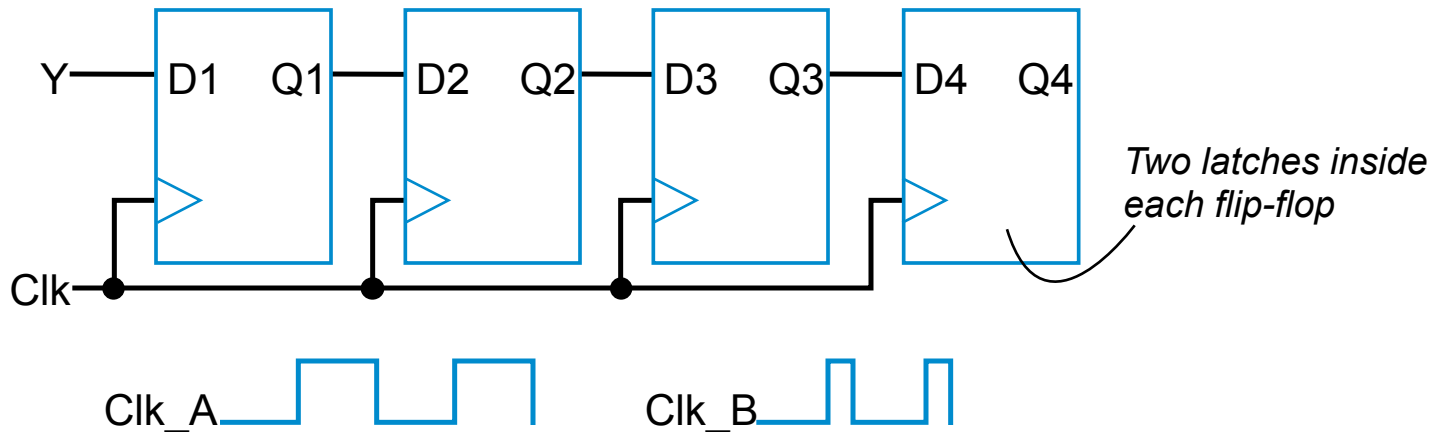
Symbol for falling-edge triggered D flip-flop

Internal design: Just invert servant clock rather than master



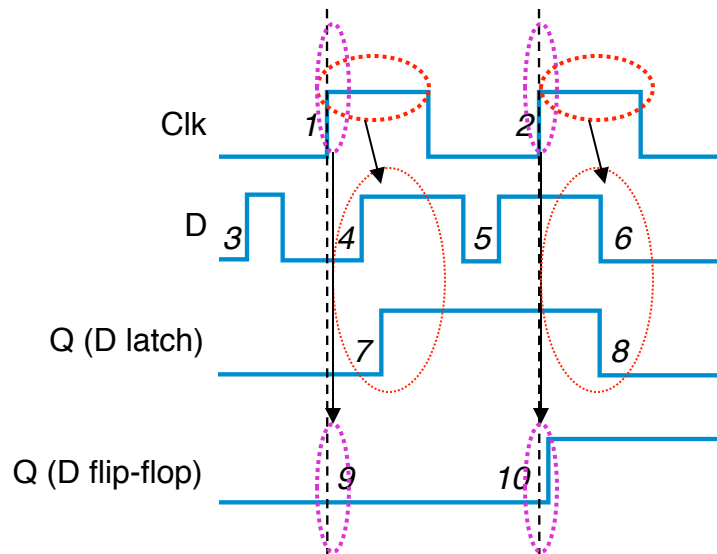
D Flip-Flop

- Solves problem of not knowing through how many latches a signal travels when $C=1$
 - In figure below, signal travels through exactly one flip-flop, for Clk_A or Clk_B
 - Why? Because on rising edge of Clk, all four flip-flops are loaded simultaneously -- then all four no longer pay attention to their input, until the next rising edge. Doesn't matter how long Clk is 1.



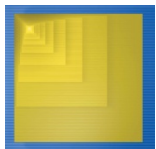
D Latch vs. D Flip-Flop

- Latch is level-sensitive
 - Stores D when C=1
- Flip-flop is edge triggered
 - Stores D when C changes from 0 to 1
- Saying “level-sensitive latch” or “edge-triggered flip-flop” is redundant
- Comparing behavior of latch and flip-flop:



*Latch follows D
while Clk is 1*

*Flip-flop only loads D
during Clk rising edge*



Flight-Attendant Call Button Using D Flip-Flop

- D flip-flop will store bit
- Inputs are Call, Cancel, and present value of D flip-flop, Q
- Truth table shown below

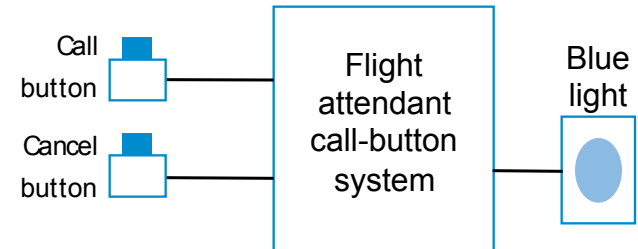
Call	Cancel	Q	D
0	0	0	0
0	0	1	1
0	1	0	0
0	1	1	0
1	0	0	1
1	0	1	1
1	1	0	1
1	1	1	1

Preserve value: if
Q=0, make D=0; if
Q=1, make D=1

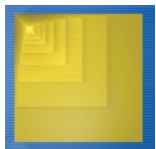
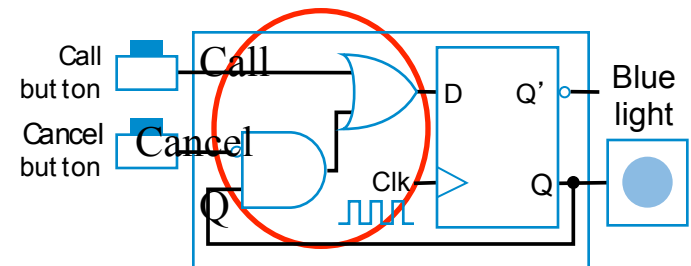
Cancel -- make
D=0

Call -- make D=1

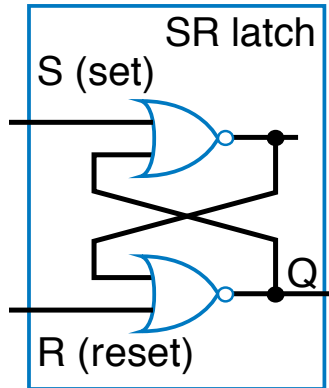
Let's give priority
to Call -- make
D=1



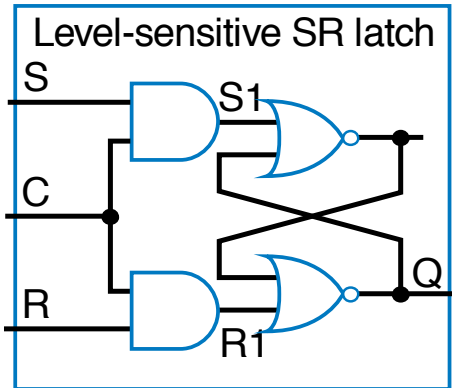
Circuit derived from truth table,
using Chapter 2 combinational
logic design process



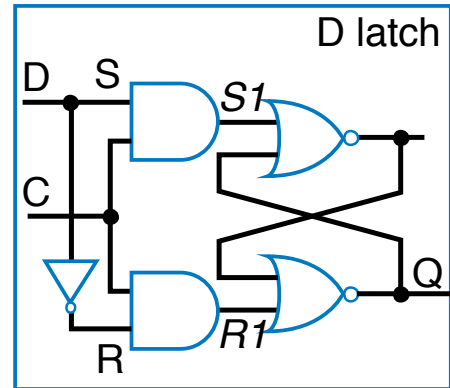
Bit Storage Summary



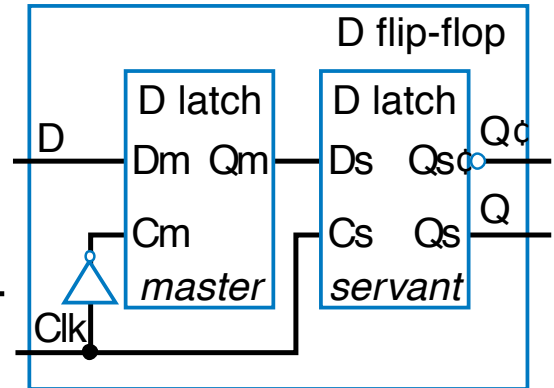
Feature: $S=1$ sets Q to 1, $R=1$ resets Q to 0.
Problem: $SR=11$ yields undefined Q , other glitches may set/reset inadvertently.



Feature: S and R only have effect when $C=1$. An external circuit can prevent $SR=11$ when $C=1$.
Problem: avoiding $SR=11$ can be a burden.



Feature: SR can't be 11.
Problem: $C=1$ for too long will propagate new values through too many latches; for too short may not result in the bit being stored.



Feature: Only loads D value present at rising clock edge, so values can't propagate to other flip-flops during same clock cycle. *Tradeoff:* uses more gates internally, and requires more external gates than SR — but transistors today are more plentiful and cheaper.

- We considered increasingly better bit storage until we arrived at the robust D flip-flop bit storage

